

CURSO PYTHON PARA INTELIGÊNCIA ARTIFICIAL

Aula 13 — Documentos: PDF + Extração de Texto

*PyPDF2 · pdfplumber · chunks · limpeza · pré-processamento para embeddings**Autor: Edie Correia Santana*

Problema

A maior parte do conhecimento corporativo está presa em PDFs: contratos, laudos, relatórios técnicos, manuais de produto, artigos científicos. Para construir um sistema de perguntas e respostas sobre esses documentos, um classificador de tópicos ou um pipeline de geração com RAG (Retrieval-Augmented Generation), o primeiro passo invariavelmente é o mesmo: **extrair o texto** do PDF de forma limpa, estruturada e reproduzível.

O desafio desta aula é construir um pipeline completo de extração: escolher a biblioteca certa para cada tipo de PDF, extrair o texto página a página, limpar o ruído introduzido pela conversão, dividir o conteúdo em chunks adequados para embeddings e exportar tudo em um JSON estruturado — o formato de entrada padrão de sistemas de IA baseados em documentos.

Objetivos de Aprendizagem

Ao final desta aula, você será capaz de:

- Distinguir PyPDF2 e pdfplumber e escolher a ferramenta adequada para cada tipo de PDF
- Extrair metadados e texto de PDFs com PyPDF2 usando a API de páginas
- Extrair texto com layout preservado, tabelas e bounding boxes com pdfplumber
- Implementar uma função de limpeza de texto que remove ruídos comuns da extração de PDFs
- Dividir texto em chunks por página, por tamanho fixo e com sobreposição (overlap)
- Aplicar pré-processamento leve para preparar texto para geração de embeddings
- Exportar o resultado da extração em JSON estruturado com metadados por página e por chunk

Conteúdo da Aula

1. PDF: Estrutura Interna e o Problema da Extração

O formato PDF (Portable Document Format) foi projetado para *apresentação visual*, não para extração de texto. Internamente, um PDF armazena instruções de renderização — posições absolutas de glifos na página, fontes embutidas, coordenadas de imagens. Não

há, em geral, um conceito de 'parágrafo' ou 'coluna' explícito no arquivo: o que parece um texto linear na tela é frequentemente uma coleção de fragmentos posicionados individualmente (ADOBE, 2023).

Isso tem implicações diretas para extração:

Tipo de PDF	Comportamento na extração
PDF nativo (texto)	Texto extraível diretamente — qualidade boa a excelente
PDF nativo com colunas	Ordem das palavras pode sair errada sem heurística de layout
PDF escaneado (imagem)	Sem texto extraível — requer OCR (coberto na Aula 14)
PDF com tabelas	Estrutura da tabela se perde sem ferramenta específica
PDF com fórmulas	Fórmulas matemáticas raramente extraem corretamente
PDF com proteção	Extração pode ser bloqueada por senha ou permissões

DICA

Antes de qualquer extração, inspecione o PDF visualmente: abra-o, tente selecionar texto com o cursor. Se conseguir selecionar, é um PDF nativo — PyPDF2 ou pdfplumber vão funcionar. Se não conseguir selecionar, é escaneado — precisará de OCR. No terminal: `pdfinfo arquivo.pdf` mostra metadados e indica se tem texto embutido.

2. PyPDF2 vs pdfplumber — Quando Usar Cada Um

Existem diversas bibliotecas Python para leitura de PDFs. As duas mais usadas em projetos de IA são **PyPDF2** (hoje mantida como **pypdf**) e **pdfplumber**. Elas têm filosofias diferentes e se complementam — conhecer os casos de uso de cada uma evita horas de tentativa e erro.

Critério	PyPDF2 / pypdf	pdfplumber
Instalação	<code>pip install pypdf</code>	<code>pip install pdfplumber</code>
Velocidade	Muito rápido	Mais lento (análise de layout)
Qualidade do texto	Boa para PDFs simples	Excelente, especialmente em PDFs complexos
Extração de tabelas	Não	Sim — <code>extract_table()</code>
Bounding boxes	Não	Sim — coordenadas x,y de cada palavra
Metadados	Sim	Sim
Manipulação (merge, split)	Sim — ponto forte	Somente leitura
Melhor para	ETL rápido, PDFs simples, manipulação	PDFs com layout, tabelas, análise precisa

Regra prática de escolha

Use **pypdf** quando precisar de velocidade, processar volumes grandes de PDFs nativos simples, ou manipular páginas (mesclar, dividir, rotacionar). Use **pdfplumber** quando a qualidade do texto extraído for crítica, quando o PDF tiver múltiplas colunas, tabelas ou quando precisar das coordenadas exatas dos elementos na página. Em caso de dúvida, teste os dois e compare o resultado visualmente.

3. Extração com pypdf

A biblioteca **pypdf** (sucessora do PyPDF2, que foi descontinuado em 2022) oferece uma API simples e eficiente para leitura de metadados e texto de PDFs nativos. É a escolha certa para pipelines de alto volume onde o layout não é crítico (PDF MAINTAINERS, 2024).

3.1 Metadados e Inspeção

```
from pypdf import PdfReader

reader = PdfReader('documento.pdf')

# — Metadados —————
meta = reader.metadata
print(f'Título:      {meta.title}')
print(f'Autor:       {meta.author}')
print(f'Criado em:    {meta.creation_date}')
print(f'Produtor:     {meta.producer}')

# — Estrutura —————
print(f'Total de páginas: {len(reader.pages)}')

# Verificar se o PDF tem senha
if reader.is_encrypted:
    print('PDF protegido por senha')
    reader.decrypt('senha_aqui') # decriptografar se necessário

# Tamanho (dimensões) da primeira página
pagina = reader.pages[0]
print(f'Largura: {pagina.mediabox.width:.1f} pt')
print(f'Altura:   {pagina.mediabox.height:.1f} pt')
```

3.2 Extração de Texto por Página

```
from pypdf import PdfReader

reader = PdfReader('documento.pdf')

# Extrair texto de uma página
texto_p1 = reader.pages[0].extract_text()
print(texto_p1[:500])

# Extrair todas as páginas
paginas = []
for i, pagina in enumerate(reader.pages):
    texto = pagina.extract_text() or '' # None → string vazia
    paginas.append({'pagina': i + 1, 'texto': texto})

print(f'{len(paginas)} páginas extraídas')
print(f'Total de caracteres: {sum(len(p["texto"]) for p in paginas)}')

# Orientação da extração — útil para PDFs com colunas
# Modos: 'default', 'retain_layout'
from pypdf.generic import ArrayObject
texto = pagina.extract_text(extraction_mode='retain_layout')
```

3.3 Manipulação de Páginas

```
from pypdf import PdfReader, PdfWriter

# Mesclar dois PDFs
writer = PdfWriter()
for caminho in ['part1.pdf', 'parte2.pdf']:
    reader = PdfReader(caminho)
    for pagina in reader.pages:
        writer.add_page(pagina)

with open('merged.pdf', 'wb') as f:
    writer.write(f)

# Extrair páginas específicas (ex: páginas 1 a 5)
writer = PdfWriter()
```

```
reader = PdfReader('documento.pdf')

for pagina in reader.pages[0:5]:      # índice 0 = página 1
    writer.add_page(pagina)

with open('paginas_1_a_5.pdf', 'wb') as f:
    writer.write(f)

print(f'Extraídas {len(writer.pages)} páginas')
```

AVISO

PyPDF2 foi descontinuado em 2022. Use sempre 'pypdf' (sem o 2) nos projetos novos. A API é quase idêntica, mas pypdf tem melhor suporte a PDFs modernos, mantém correções de segurança ativas e recebe atualizações regulares. Instale com: `pip install pypdf`

4. Extração com pdfplumber

O **pdfplumber** é construído sobre a biblioteca **pdfminer.six** e oferece acesso de baixo nível ao layout do PDF: cada palavra tem coordenadas *x/y*, fonte, tamanho e cor. Isso permite reconstruir a ordem de leitura correta em documentos complexos e extrair tabelas estruturadas — algo que o pypdf simplesmente não faz (SINGER-VINE, 2024).

4.1 Extração de Texto com Contexto

```
import pdfplumber

with pdfplumber.open('documento.pdf') as pdf:

    # Metadados
    print(pdf.metadata)
    print(f'{len(pdf.pages)} páginas')

    # Extrair texto de uma página
    pagina = pdf.pages[0]
    texto = pagina.extract_text()
    print(texto[:500])

    # extract_text() com separador de layout
    # x_tolerance: distância horizontal que define espaço entre palavras
    # y_tolerance: distância vertical que define nova linha
```

```
texto = pagina.extract_text(x_tolerance=3, y_tolerance=3)

# Todas as páginas
paginas = []

for i, pag in enumerate(pdf.pages):
    texto = pag.extract_text() or ''
    paginas.append({'pagina': i + 1, 'texto': texto})
```

4.2 Extração de Tabelas

O **extract_table()** do **pdfplumber** detecta tabelas automaticamente usando heurísticas de layout (linhas horizontais e verticais, espaçamento de colunas) e retorna uma lista de listas — equivalente a uma tabela bidimensional:

```
import pdfplumber
import pandas as pd

with pdfplumber.open('relatorio.pdf') as pdf:
    pagina = pdf.pages[0]

    # Extrair primeira tabela da página
    tabela = pagina.extract_table()

    if tabela:
        # Primeira linha como cabeçalho
        df = pd.DataFrame(tabela[1:], columns=tabela[0])
        print(df)

    # Extrair todas as tabelas de uma página
    tabelas = pagina.extract_tables()
    print(f'{len(tabelas)} tabela(s) na página 1')

    # Iterar por todas as páginas buscando tabelas
    for i, pag in enumerate(pdf.pages):
        for j, tabela in enumerate(pag.extract_tables()):
            df = pd.DataFrame(tabela[1:], columns=tabela[0])
            df.to_csv(f'tabela_p{i+1}_t{j+1}.csv', index=False)
            print(f'Tabela {j+1} da página {i+1}: {df.shape}')
```

4.3 Bounding Boxes e Palavras

O acesso às palavras individuais com suas coordenadas é o recurso mais poderoso do **pdfplumber** para diagnóstico e extração precisa:

```
import pdfplumber

with pdfplumber.open('documento.pdf') as pdf:
    pagina = pdf.pages[0]

    # Lista de palavras com metadados completos
    palavras = pagina.extract_words()
    for palavra in palavras[:5]:
        print(palavra)

    # {'text': 'Relatório', 'x0': 72.0, 'y0': 741.0,
    #  'x1': 145.3, 'y1': 754.0, 'fontname': 'Arial-Bold', ...}

    # Recortar região específica da página (bounding box)
    # Útil para extrair apenas o corpo, ignorando cabeçalho e rodapé
    # Coordenadas: (x0, top, x1, bottom) em pontos
    altura = float(pagina.height)
    corpo = pagina.crop((0, 80, pagina.width, altura - 80))
    texto_corpo = corpo.extract_text() or ''
    print(f'Texto do corpo: {len(texto_corpo)} caracteres')

    # Remover cabeçalho e rodapé de todas as páginas
    textos_limpos = []
    for pag in pdf.pages:
        recorte = pag.crop((0, 60, pag.width, float(pag.height) - 60))
        textos_limpos.append(recorte.extract_text() or '')
```

DICA

Use `pdfplumber.open()` sempre como gerenciador de contexto ('with'). Ele mantém o arquivo aberto enquanto o bloco está em execução e fecha automaticamente ao sair, liberando memória e handles do sistema operacional. Para PDFs grandes (>200 páginas), processe uma página por vez dentro do 'with' em vez de carregar tudo na memória.

5. Limpeza de Texto Extraído

O texto bruto extraído de PDFs raramente está pronto para uso. A extração introduz ruídos sistemáticos: hifenização de palavras que quebraram no fim da linha, múltiplos espaços, quebras de linha no meio de frases, cabeçalhos e rodapés repetidos em cada página, caracteres especiais de fontes proprietárias. Uma função de limpeza robusta é parte

obrigatória de qualquer pipeline de processamento de documentos (LANE; HOWARD; HAPKE, 2019).

5.1 Padrões de Ruído Mais Comuns

Problema	Exemplo no texto bruto	Após limpeza
Hifenização de quebra	aná-\nlise de dados	análise de dados
Quebra no meio de frase	o contrato\nprevê	o contrato prevê
Múltiplos espaços	dados inconsistentes	dados inconsistentes
Linhas em branco excessivas	\n\n\n\n	\n\n (máx 2)
Cabeçalhos repetidos	Página 1 de 10\nEmpresa XYZ	(removido)
Caracteres de controle	\x0c (form feed)	(removido)
Bullet points corrompidos	\uf0b7 item	• item

5.2 Função de Limpeza

```
import re
import unicodedata

def limpar_texto(texto: str) -> str:
    """
    Aplica pipeline de limpeza em texto extraído de PDF.
    Retorna string limpa, pronta para chunking ou embeddings.
    """
    if not texto or not texto.strip():
        return ''

    # 1. Normalizar unicode (NFC): unificar formas de caracteres acentuados
    texto = unicodedata.normalize('NFC', texto)

    # 2. Remover caracteres de controle (\x00-\x1F exceto \n e \t)
    texto = re.sub(r'[\x00-\x08\x0b-\x1f\x7f]', '', texto)

    # 3. Reparar hifenização de quebra de linha
    # Ex: 'aná-\nlise' -> 'análise'
    texto = re.sub(r'(\w+)-\n(\w+)', r'\1\2', texto)

    # 4. Juntar linhas quebradas no meio de frases
```



```
# Uma quebra de linha isolada (sem linha em branco antes) → espaço
texto = re.sub(r'(?!\n)\n(?!\n)', ' ', texto)

# 5. Recolher múltiplos espaços em um único
texto = re.sub(r'[ \t]+', ' ', texto)

# 6. Limitar linhas em branco consecutivas a no máximo 2
texto = re.sub(r'\n{3,}', '\n\n', texto)

# 7. Limpar espaço antes de pontuação
texto = re.sub(r' +([.,;:!?])', r'\1', texto)

return texto.strip()

# Teste rápido
bruto = 'aná-\nlise de da-\ndos\n\n\n\n  Página 1 de 10  '
print(repr(limpar_texto(bruto)))

# 'análise de dados'
```

6. Dividindo em Chunks

Modelos de linguagem têm uma janela de contexto limitada — o GPT-4 aceita até ~128k tokens, mas embeddings como o text-embedding-3-small têm limite de 8 192 tokens. Além disso, chunks menores permitem recuperação mais precisa em sistemas de RAG: ao buscar pelo trecho mais relevante, um chunk de 500 tokens é mais específico do que um capítulo inteiro. A estratégia de chunking é uma das decisões de engenharia mais importantes em pipelines de documentos (GARG, 2024).

6.1 Chunk por Página

A forma mais simples: cada página do PDF vira um chunk. Adequada quando as páginas são semanticamente coesas (relatórios, artigos com seções por página):

```
def chunks_por_pagina(paginas: list[dict]) -> list[dict]:
    """
    Recebe lista de {'pagina': N, 'texto': '...'}
    Retorna a mesma lista, pulando páginas vazias.
    """
    return [
```

```
{
    'chunk_id': f'p{p["pagina"]:03d}',
    'pagina': p['pagina'],
    'texto': limpar_texto(p['texto']),
    'n_chars': len(p['texto']),
}

for p in paginas
    if p['texto'].strip()

]
```

6.2 Chunk por Tamanho Fixo com Overlap

Divide o texto em blocos de *tamanho_max* caracteres com **sobreposição** (overlap) entre chunks consecutivos. O overlap garante que frases que caem na fronteira entre dois chunks não sejam perdidas durante a recuperação — uma das técnicas mais usadas em sistemas RAG:

```
def chunks_por_tamanho(
    texto: str,
    tamanho_max: int = 1000,
    overlap: int = 200,
    origem: str = '',
) -> list[dict]:
    """
    Divide texto em chunks de até tamanho_max caracteres.
    Chunks consecutivos se sobrepõem em 'overlap' caracteres.
    """
    texto = limpar_texto(texto)
    if not texto:
        return []

    chunks = []
    inicio = 0
    idx = 0

    while inicio < len(texto):
        fim = inicio + tamanho_max
        trecho = texto[inicio:fim]

        # Tentar quebrar no espaço mais próximo antes do limite
```

```
# para não cortar palavras ao meio
if fim < len(texto):
    ultimo_espaco = trecho.rfind(' ')
    if ultimo_espaco > tamanho_max * 0.6: # ao menos 60% preenchido
        trecho = trecho[:ultimo_espaco]
        fim = inicio + ultimo_espaco

chunks.append({
    'chunk_id': f'{origem}_c{idx:03d}' if origem else f'c{idx:03d}',
    'texto':    trecho.strip(),
    'n_chars':  len(trecho.strip()),
    'inicio':   inicio,
    'fim':      inicio + len(trecho),
})

chunk    inicio = fim - overlap # retroceder 'overlap' chars para próximo
        idx    += 1

return chunks
```

6.3 Chunk por Parágrafo

Quando o texto tem estrutura clara de parágrafos (duas quebras de linha como delimitador), dividir por parágrafo produz chunks semanticamente mais coesos do que por tamanho fixo:

```
def chunks_por_paragrafo(
    texto: str,
    max_chars: int = 2000,
    origem: str = '',
) -> list[dict]:
    """
    Divide por parágrafo (\n\n). Agrega parágrafos curtos até
    atingir max_chars para evitar chunks muito pequenos.
    """
    texto = limpar_texto(texto)
    paragrafos = [p.strip() for p in texto.split('\n\n') if p.strip()]

    chunks, atual, idx = [], '', 0
```

```
for par in paragrafos:
    candidato = (atual + '\n\n' + par).strip() if atual else par
    if len(candidato) <= max_chars:
        atual = candidato
    else:
        if atual:
            chunks.append({
                'chunk_id': f'{origem}_c{idx:03d}' if origem else
f'c{idx:03d}',
                'texto': atual,
                'n_chars': len(atual),
            })
            idx += 1
        atual = par
    if atual:
        chunks.append({'chunk_id': f'{origem}_c{idx:03d}' if origem else
f'c{idx:03d}',
                        'texto': atual, 'n_chars': len(atual)})

return chunks
```

DICA

Não existe tamanho de chunk universalmente ideal. A regra prática:

- Chunks de 256–512 tokens ($\approx$ 1 000–2 000 chars): boa precisão na recuperação RAG.
 - Chunks de 1 000–2 000 tokens: melhor contexto por chunk, menor precisão de busca.
 - Overlap de 10–20% do tamanho do chunk evita perda de informação nas bordas.
- Sempre registre o tamanho e o método de chunking no JSON de saída para reprodutibilidade.

7. Pré-processamento para Embeddings

Embeddings são representações vetoriais de texto geradas por modelos de linguagem. Antes de enviar um chunk para um modelo de embedding (OpenAI, Sentence-BERT, etc.), um pré-processamento leve melhora a qualidade da representação e reduz tokens desnecessários. O objetivo **não** é fazer NLP clássico pesado (stemming, remoção de stopwords) — isso remove informação semântica relevante para embeddings modernos. O objetivo é apenas normalizar e limpar o texto (REIMERS; GUREVYCH, 2019).

7.1 Pipeline de Pré-processamento Leve

```
import re
import unicodedata
```

```
def preprocessar_para_embedding(texto: str) -> str:
    """
    Pré-processamento leve para texto destinado a embeddings.
    NÃO remove stopwords nem faz stemming — isso prejudica embeddings.
    """
    if not texto or not texto.strip():
        return ''

    # 1. Limpeza base
    texto = limpar_texto(texto)

    # 2. Lowercase — opcional; depende do modelo de embedding
    # Modelos case-sensitive (BERT, E5) performam melhor sem lowercase
    # Modelos antigos (word2vec) beneficiam do lowercase
    # texto = texto.lower() # descomente apenas se necessário

    # 3. Remover URLs
    texto = re.sub(r'https?:\/\/\S+|www\.\S+', '[URL]', texto)

    # 4. Remover e-mails
    texto = re.sub(r'\b[\w.+~]+@[~\w-]+\.[\w.]+\b', '[EMAIL]', texto)

    # 5. Normalizar números longos (evita tokens raros)
    # Preservar anos (4 dígitos) e valores monetários com contexto
    texto = re.sub(r'\b\d{6,}\b', '[NUM]', texto)

    # 6. Garantir espaço único
    texto = re.sub(r'[ \t]+', ' ', texto)

    return texto.strip()

# Estimativa de tokens (regra dos 4 chars por token para português)
def estimar_tokens(texto: str, chars_por_token: float = 4.0) -> int:
    return max(1, int(len(texto) / chars_por_token))
```

```
# Teste

texto = 'Acesse https://empresa.com ou envie para joao@empresa.com.br.'
print(preprocessar_para_embedding(texto))

# 'Acesse [URL] ou envie para [EMAIL].'
```

7.2 Estrutura do JSON de Saída

O formato de saída padrão para pipelines de extração é um JSON com metadados do documento na raiz e uma lista de chunks, cada um com seu **chunk_id** único, texto, metadados de origem e estimativa de tokens. Esse formato é a entrada direta de sistemas de vectorstore como Chroma, FAISS e Pinecone:

```
{
  "documento": {
    "arquivo": "relatorio_anual.pdf",
    "n_paginas": 42,
    "titulo": "Relatório Anual 2024",
    "autor": "Departamento Financeiro",
    "extraido_em": "2025-03-01T10:22:05",
    "biblioteca": "pdfplumber",
    "metodo_chunk": "tamanho_fixo",
    "chunk_size": 1000,
    "chunk_overlap": 200
  },
  "chunks": [
    {
      "chunk_id": "relatorio_anual_c000",
      "pagina": 1,
      "texto": "Este relatório apresenta os resultados...",
      "n_chars": 987,
      "n_tokens_est": 247
    },
    {
      "chunk_id": "relatorio_anual_c001",
      "pagina": 1,
      "texto": "...consolidados do exercício fiscal de 2024...",
      "n_chars": 1002,
      "n_tokens_est": 251
    }
  ]
}
```

```
}  
]  
}
```

Mini-lab: Extrair Texto de PDF e Gerar JSON de Chunks

Chegou o momento de integrar todos os conceitos desta aula em um pipeline real. Você vai construir o script **extrator_pdf.py** que aceita qualquer PDF nativo como entrada, extrai o texto com **pdfplumber**, limpa, divide em chunks e exporta um JSON estruturado — pronto para ser usado como entrada de um sistema de embeddings ou RAG.

Objetivo

Construir um script de linha de comando completo que: inspeciona o PDF, extrai texto página a página com **pdfplumber**, aplica o pipeline de limpeza, gera chunks com overlap, calcula estatísticas por página e por chunk, e exporta dois arquivos — **paginas.json** (mapeamento página → texto limpo) e **chunks.json** (chunks prontos para embeddings).

Estrutura do Projeto

```
aula13/  
├── .venv/  
├── extrator_pdf.py    ← script principal  
├── documento.pdf     ← qualquer PDF nativo para testar  
├── paginas.json       ← gerado pelo script  
└── chunks.json        ← gerado pelo script
```

Setup

```
mkdir -p ~/projetos/aula13 && cd ~/projetos/aula13  
python -m venv .venv && source .venv/bin/activate  
pip install pdfplumber pypdf  
  
# Qualquer PDF nativo serve para testar. Para criar um de exemplo:  
# Copie um PDF qualquer do seu sistema para a pasta aula13/  
# cp ~/Downloads/algum_relatorio.pdf documento.pdf  
  
touch extrator_pdf.py
```

Código Completo — extrator_pdf.py

```
# extrator_pdf.py
```

```
# Mini-lab Aula 13 — Extração de PDF e geração de JSON de chunks

from __future__ import annotations

import json
import re
import sys
import unicodedata
from datetime import datetime
from pathlib import Path

import pdfplumber
from pypdf import PdfReader

# — Limpeza —————
def limpar_texto(texto: str) -> str:
    if not texto or not texto.strip():
        return ''

    texto = unicodedata.normalize('NFC', texto)
    texto = re.sub(r'[\x00-\x08\x0b-\x1f\x7f]', '', texto)
    texto = re.sub(r'(\w+)-\n(\w+)', r'\1\2', texto)
    texto = re.sub(r'(?!\n)\n(?!\\n)', ' ', texto)
    texto = re.sub(r'[ \t]+', ' ', texto)
    texto = re.sub(r'\n{3,}', '\n\n', texto)
    texto = re.sub(r'+([.,;:!?])', r'\1', texto)
    return texto.strip()

def preprocessar_para_embedding(texto: str) -> str:
    texto = limpar_texto(texto)
    texto = re.sub(r'https?:\/\/\S+|www\.\S+', '[URL]', texto)
    texto = re.sub(r'\b[\w.+~]+@[~\w-]+\.[\w.]+\b', '[EMAIL]', texto)
    texto = re.sub(r'\b\d{6,}\b', '[NUM]', texto)
    return re.sub(r'[ \t]+', ' ', texto).strip()
```



```
def estimar_tokens(texto: str) -> int:
    return max(1, int(len(texto) / 4))

# --- Chunking ---
def chunks_por_tamanho(
    texto: str,
    tamanho_max: int = 1000,
    overlap: int = 200,
    origem: str = '',
) -> list[dict]:
    texto = limpar_texto(texto)
    if not texto:
        return []

    chunks, inicio, idx = [], 0, 0
    while inicio < len(texto):
        fim = inicio + tamanho_max
        trecho = texto[inicio:fim]
        if fim < len(texto):
            ult = trecho.rfind(' ')
            if ult > tamanho_max * 0.6:
                trecho = trecho[:ult]
                fim = inicio + ult
        trecho = trecho.strip()
        if trecho:
            chunks.append({
                'chunk_id': f'{origem}_c{idx:03d}' if origem else
f'c{idx:03d}',
                'texto': trecho,
                'texto_embed': preprocessar_para_embedding(trecho),
                'n_chars': len(trecho),
                'n_tokens_est': estimar_tokens(trecho),
                'inicio_char': inicio,
            })
            idx += 1
```

```
        inicio = fim - overlap
    return chunks

# --- Extração ---
def extrair_pdf(
    caminho: str | Path,
    tamanho_chunk: int = 1000,
    overlap: int = 200,
) -> dict:
    """
    Pipeline completo: inspecionar → extrair → limpar → chunkar.
    Retorna dicionário com 'documento', 'paginas' e 'chunks'.
    """
    caminho = Path(caminho)
    if not caminho.exists():
        raise FileNotFoundError(f'Arquivo não encontrado: {caminho}')

    # --- Metadados via pypdf ---
    reader = PdfReader(str(caminho))
    meta = reader.metadata or {}
    n_pags = len(reader.pages)

    doc_meta = {
        'arquivo': caminho.name,
        'n_paginas': n_pags,
        'titulo': getattr(meta, 'title', None) or caminho.stem,
        'autor': getattr(meta, 'author', None) or 'Desconhecido',
        'produtor': getattr(meta, 'producer', None) or '',
        'extraido_em': datetime.now().isoformat(timespec='seconds'),
        'biblioteca': 'pdfplumber',
        'metodo_chunk': 'tamanho_fixo',
        'chunk_size': tamanho_chunk,
        'chunk_overlap': overlap,
    }
```

```
# — Extração página a página com pdfplumber —————
paginas: list[dict] = []
chunks_total: list[dict] = []
origem = caminho.stem

with pdfplumber.open(str(caminho)) as pdf:
    for i, pag in enumerate(pdf.pages):
        num_pag = i + 1
        texto_bruto = pag.extract_text() or ''
        texto_limpo = limpar_texto(texto_bruto)

        paginas.append({
            'pagina': num_pag,
            'texto': texto_limpo,
            'n_chars': len(texto_limpo),
            'n_tokens_est': estimar_tokens(texto_limpo),
            'vazia': len(texto_limpo.strip()) == 0,
        })

# Gerar chunks apenas para páginas com conteúdo
if texto_limpo.strip():
    chunks_pag = chunks_por_tamanho(
        texto_limpo,
        tamanho_max=tamanho_chunk,
        overlap=overlap,
        origem=f'{origem}_p{num_pag:03d}',
    )
    # Anotar a página de origem em cada chunk
    for chunk in chunks_pag:
        chunk['pagina'] = num_pag
    chunks_total.extend(chunks_pag)

return {
    'documento': doc_meta,
    'paginas': paginas,
    'chunks': chunks_total,
```

```
}

# — Saída e estatísticas —————
def imprimir_resumo(resultado: dict) -> None:
    doc    = resultado['documento']
    pags    = resultado['paginas']
    chunks = resultado['chunks']

    sep = '=' * 55
    print(f'\n{sep}')
    print(f'  EXTRAÇÃO DE PDF')
    print(sep)
    print(f'  Arquivo   : {doc["arquivo"]}')
    print(f'  Título    : {doc["titulo"]}')
    print(f'  Autor     : {doc["autor"]}')
    print(f'  Páginas   : {doc["n_paginas"]}')
    print(f'  Extraído  : {doc["extraido_em"]}')

    vazias = sum(1 for p in pags if p['vazia'])
    total_chars = sum(p['n_chars'] for p in pags)
    total_tokens = sum(p['n_tokens_est'] for p in pags)
    print(f'\n  Páginas com texto : {doc["n_paginas"] - vazias}')
    print(f'  Páginas vazias    : {vazias}')
    print(f'  Total de chars    : {total_chars:,}')
    print(f'  Tokens estimados  : {total_tokens:,}')

    print(f'\n  Chunks gerados    : {len(chunks)}')
    if chunks:
        tamanhos = [c['n_chars'] for c in chunks]
        print(f'  Chunk mín/méd/máx : {min(tamanhos)} / {sum(tamanhos)//len(tamanhos)} / {max(tamanhos)} chars')
        print(f'  Tokens mín/méd/máx: {min(c["n_tokens_est"] for c in chunks)} /',
              f'{sum(c["n_tokens_est"] for c in chunks)//len(chunks)} /',
              f'{max(c["n_tokens_est"] for c in chunks)}')
```

```
# Prévia dos primeiros 3 chunks

print(f'\n  Prévia (primeiros 3 chunks):')

for c in chunks[:3]:
    preview = c['texto'][:120].replace('\n', ' ')
    print(f'    [{c["chunk_id"]}]\t({c["n_chars"]} chars, p.{c["pagina"]})')
    print(f'        {preview}...')

print(f'\n{sep}\n')

def salvar_json(dados: dict, caminho: str | Path) -> None:
    with open(caminho, 'w', encoding='utf-8') as f:
        json.dump(dados, f, ensure_ascii=False, indent=2)
    print(f'  Salvo: {caminho}')

# — Ponto de entrada —————

def main() -> None:
    if len(sys.argv) < 2:
        print('Uso: python extrator_pdf.py <arquivo.pdf> [chunk_size]
[overlap]')
        print('Ex:  python extrator_pdf.py documento.pdf 1000 200')
        sys.exit(1)

    caminho      = Path(sys.argv[1])
    chunk_size   = int(sys.argv[2]) if len(sys.argv) > 2 else 1000
    overlap      = int(sys.argv[3]) if len(sys.argv) > 3 else 200

    print(f'Processando: {caminho.name} ...')
    resultado = extrair_pdf(caminho, chunk_size, overlap)

    imprimir_resumo(resultado)

    # Salvar paginas.json (mapeamento página → texto)
    saida_paginas = caminho.stem + '_paginas.json'
    salvar_json({
```

```
        'documento': resultado['documento'],
        'paginas':    resultado['paginas'],
    }, saida_paginas)

# Salvar chunks.json (chunks prontos para embeddings)
saida_chunks = caminho.stem + '_chunks.json'
salvar_json({
    'documento': resultado['documento'],
    'chunks':    resultado['chunks'],
}, saida_chunks)

print(f'Concluído! {len(resultado["chunks"])} chunks exportados.')

if __name__ == '__main__':
    main()
```

Execução

```
# Modo padrão: chunks de 1 000 chars com overlap de 200
(.venv) $ python extrator_pdf.py documento.pdf

# Chunks menores para embeddings (mais precisão na busca RAG)
(.venv) $ python extrator_pdf.py documento.pdf 600 100

# Chunks grandes para modelos com contexto longo
(.venv) $ python extrator_pdf.py documento.pdf 2000 400
```

Saída Esperada (exemplo com PDF de 10 páginas)

```
Processando: documento.pdf ...

=====
EXTRAÇÃO DE PDF
=====

Arquivo   : documento.pdf
Título    : Relatório Técnico Q1 2025
Autor     : Departamento de Engenharia
```

```
Páginas : 10
Extraído : 2025-03-01T14:33:07

Páginas com texto : 9
Páginas vazias : 1
Total de chars : 38.420
Tokens estimados : 9.605

Chunks gerados : 42
Chunk mín/méd/máx : 212 / 891 / 1000 chars
Tokens mín/méd/máx: 53 / 223 / 250

Prévia (primeiros 3 chunks):
[documento_p001_c000] (998 chars, p.1)
    Este relatório apresenta os resultados consolidados do primeiro...
[documento_p001_c001] (743 chars, p.1)
    ...trimestre de 2025, incluindo análise de desempenho por área...
[documento_p002_c000] (1000 chars, p.2)
    A receita líquida atingiu R$ 42,3 milhões no período, representando...

=====

Salvo: documento_paginas.json
Salvo: documento_chunks.json
Concluído! 42 chunks exportados.
```

Inspecionar o JSON gerado

```
# Ver estrutura do JSON de páginas
python -c "
import json
d = json.load(open('documento_paginas.json'))
print('Páginas:', len(d['paginas']))
print('Primeira página (prévia):')
print(d['paginas'][0]['texto'][:200])
"
```

```
# Ver estrutura do JSON de chunks

python -c "

import json

d = json.load(open('documento_chunks.json'))

print('Total chunks:', len(d['chunks']))

print('Primeiro chunk:')

import pprint; pprint.pprint(d['chunks'][0])

"
```

DICA

O campo 'texto_embed' de cada chunk já tem URLs e e-mails substituídos por tokens neutros ([URL], [EMAIL]). Use esse campo — não o 'texto' bruto — ao chamar a API de embeddings. O campo 'texto' preserva o conteúdo original para exibição ao usuário quando o chunk for recuperado pelo sistema de busca.

Desafios Extras

1. Adicione ao script a detecção automática de PDF escaneado: se a média de **n_chars** por página for menor que 100, imprima um aviso '*PDF possivelmente escaneado — considere OCR (Aula 14)*' e registre isso no JSON de saída.
2. Implemente a função **chunks_por_secao(texto)** que detecta títulos de seção usando a heurística de linhas curtas em maiúsculas (ex: '*1. INTRODUÇÃO*') e usa-os como delimitadores de chunks, em vez de tamanho fixo.
3. Adicione extração de tabelas ao pipeline: para cada página, chame **pag.extract_tables()**, converta cada tabela para um DataFrame com pandas e serialize como '**tabelas**' no JSON de saída junto às páginas.
4. Transforme o script em um módulo reutilizável usando o padrão da Aula 07: mova as funções para **src/extrator/**, crie **__init__.py** com API pública e mantenha o **__main__.py** como ponto de entrada de linha de comando.

Referências

ADOBE SYSTEMS. PDF reference, sixth edition: Adobe Portable Document Format version 1.7. San Jose: Adobe Systems, 2023. Disponível em: <https://opensource.adobe.com/dc-acrobat-sdk-docs/>. Acesso em: 20 mar. 2026.

GARG, Naman. Chunking strategies for LLM applications. 2024. Disponível em: <https://www.pinecone.io/learn/chunking-strategies/>. Acesso em: 20 mar. 2026.

LANE, Hobson; HOWARD, Cole; HAPKE, Hannes Max. Natural language processing in action. Shelter Island: Manning Publications, 2019.

PDF MAINTAINERS. pypdf documentation. 2024. Disponível em: <https://pypdf.readthedocs.io/>. Acesso em: 20 mar. 2026.

REIMERS, Nils; GUREVYCH, Iryna. Sentence-BERT: sentence embeddings using siamese BERT-networks. In: CONFERENCE ON EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING, 2019, Hong Kong. Proceedings... Hong Kong: EMNLP, 2019. p. 3982–3992.

SINGER-VINE, Jeremy. pdfplumber documentation. 2024. Disponível em: <https://github.com/jsvine/pdfplumber>. Acesso em: 20 mar. 2026.